

Slide Bài Giảng Seminar T10 — Mutation Testing & Test Effectiveness

Môn học: CS423 / CSC15003 — Kiểm thử phần mềm (FIT@HCMUS)

GVHD: ThS. Hồ Tuấn Thanh

Nhóm 08:

- 23127195 — Trần Mạnh Hùng
- 23127060 — Ninh Văn Khải
- 23127259 — Nguyễn Tấn Thắng

Slide 1: Giới Thiệu Đề Tài

Người phụ trách: Cả Nhóm 08 (23127195 - Trần Mạnh Hùng · 23127060 - Ninh Văn Khải · 23127259 - Nguyễn Tấn Thắng)

Seminar T10: Mutation Testing & Test Effectiveness

- Câu hỏi đặt ra cho bài học hôm nay:

"Bạn đã viết test, test đang xanh hết. Vậy làm sao biết test đó có thực sự kiểm tra đúng không?"

- Mục tiêu bài giảng:

1. Giải thích tại sao "test xanh" chưa phải là test tốt.
2. Giới thiệu kỹ thuật **Mutation Testing** để đo chất lượng bộ test.
3. Demo thực tế trên ứng dụng web EShop bằng công cụ Stryker.

Slide 2: Vấn Đề Với Bài Test Của Chúng Ta Hôm Nay

Người phụ trách: 23127195 - Trần Mạnh Hùng

Một bài test "xanh" chưa chắc là bài test tốt

Hãy xem ví dụ đơn giản sau:

Hàm kiểm tra người dùng có đủ 18 tuổi không:

```
function isAdult(age) {  
  return age ≥ 18;  
}
```

Bài test được viết:

```
test('kiểm tra người lớn', () => {  
  expect(isAdult(20)).toBe(true);  
});
```

Kết quả: Test XANH, báo 100% coverage.

Nhưng nếu lập trình viên vô tình đổi $\geq$ thành $>$, bài test đó **vẫn xanh**. Người 18 tuổi sẽ bị từ chối oan — và không có test nào phát hiện ra!

Slide 3: Mutation Testing Là Gì? (Giải Thích Bằng Hình Ảnh)

Người phụ trách: 23127195 - Trần Mạnh Hùng

Ý tưởng: Đóng vai người phá hoại

Hãy tưởng tượng:

- **Bạn là lập trình viên.** Bạn viết code và viết test.
- **Công cụ Mutation Testing** đóng vai một "người phá hoại" — nó tự động sửa một lỗi nhỏ vào code của bạn.
- Sau đó nó chạy lại toàn bộ bộ test và hỏi: "**Bộ test của bạn có phát hiện ra lỗi này không?**"

Hai kết quả có thể xảy ra:

Tình huống	Ý nghĩa
Bộ test bị FAIL → Mutant bị diệt (Killed)	Bộ test đủ tốt để bắt lỗi
Bộ test vẫn PASS → Mutant sống sót (Survived)	Bộ test có lỗ hổng, cần cải thiện

Slide 4: "Mutant" Là Gì? Ví Dụ Cụ Thể

Người phụ trách: 23127195 - Trần Mạnh Hùng

Công cụ thay đổi một chút nhỏ trong code

Code gốc:

```
if (age ≥ 18) {  
    return "Được phép";  
}
```

Mutant 1 (đổi dấu so sánh):

```
if (age > 18) {    // ≥ đổi thành >  
    return "Được phép";  
}
```

Mutant 2 (đổi điều kiện logic):

```
if (age ≤ 18) {    // ≥ đổi thành ≤  
    return "Được phép";  
}
```

Slide 5: Điểm Mutation Score Là Gì?

Người phụ trách: 23127195 - Trần Mạnh Hùng

Cách tính điểm chất lượng bộ test

$$\text{Mutation Score} = (\text{Số Mutant bị diệt}) / (\text{Tổng số Mutant}) \times 100\%$$

Ví dụ:

- Công cụ tạo ra 100 mutant.
- Bộ test phát hiện và diệt được 75 mutant.
- Mutation Score = **75%**

Ý nghĩa của các mức điểm:

Mức điểm	Đánh giá
Dưới 40%	Bộ test rất yếu
40% - 70%	Bộ test trung bình
70% - 90%	Bộ test tốt
Trên 90%	Bộ test rất chặt chẽ

Slide 6: 4 Trạng Thái Của Một Mutant

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Sau khi chạy bộ test với mutant, có 4 kết quả có thể xảy ra

Trạng thái	Biểu tượng	Giải thích
Killed (Diệt)		Bộ test phát hiện được lỗi — bộ test đang hoạt động tốt
Survived (Sống sót)		Bộ test không phát hiện được lỗi — cần viết thêm test
Timeout		Mutant gây ra vòng lặp vô tận, bộ test không kết thúc được — cũng được tính là diệt
NoCoverage		Không có test case nào chạy đến dòng code bị sửa — test thiếu hoàn toàn

Điều cần nhớ: Chỉ Survived mới là vấn đề thực sự cần sửa.

Slide 7: Công Cụ Nào Làm Được Việc Này?

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Có nhiều công cụ Mutation Testing cho từng ngôn ngữ

Ngôn ngữ	Công cụ phổ biến
JavaScript / TypeScript	Stryker Mutator
Java	PITest
Python	mutmut
C / C++	mull

Nhóm dùng Stryker vì:

- Ứng dụng EShop được viết bằng **Node.js + JavaScript**.
- Stryker hỗ trợ chạy cùng với **Jest** (framework test phổ biến của JavaScript).
- Stryker xuất báo cáo dạng **HTML trực quan**, dễ đọc và phân tích.

Slide 8: Bài Test Là Gì? Jest Là Gì?

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Giải thích nhanh về Jest và unit test

Jest là một công cụ cho phép bạn viết bài kiểm tra tự động cho code JavaScript.

Ví dụ một bài test bằng Jest:

```
// Hàm cần kiểm tra:
function tinh_tong(a, b) {
  return a + b;
}

// Bài test:
test('tính tổng hai số', () => {
  const ket_qua = tinh_tong(3, 4);
  expect(ket_qua).toBe(7); // Kỳ vọng kết quả phải là 7
});
```

Giải thích dòng `expect( ... ).toBe( ... )`:

- `expect(ket_qua)` — Đây là kết quả thực tế từ hàm.
- `.toBe(7)` — Đây là kết quả kỳ vọng.
- Nếu hai cái không khớp nhau → Test FAIL → Mutant bị diệt.

Slide 9: Tại Sao Bộ Test "Yếu" Không Diệt Được Mutant?

Người phụ trách: 23127060 - Ninh Văn Khải

Test yếu = chỉ kiểm tra "có chạy không", không kiểm tra "đúng không"

Ví dụ bộ test yếu:

```
test('API đăng nhập', async () => {  
  const res = await fetch('/api/login', { ... });  
  expect(res.status).toBe(200); // Chỉ kiểm tra code trạng thái HTTP  
});
```

Mutant sửa logic bên trong hàm đăng nhập → API vẫn trả về status 200 → Test vẫn PASS → Mutant sống sót.

Cách khắc phục — Viết test chặt hơn:

```
test('API đăng nhập', async () => {  
  const res = await fetch('/api/login', { ... });  
  expect(res.status).toBe(200);  
  expect(res.body.token).toBeDefined(); // Kiểm tra có token không  
  expect(res.body.user.email).toBe('test@email.com'); // Kiểm tra email đúng  
});
```

Slide 10: Demo Thực Tế — Chạy Stryker Trên Ứng Dụng EShop

Người phụ trách: 23127060 - Ninh Văn Khải

Stryker hoạt động như thế nào khi chạy?

Bước 1: Nhóm gõ lệnh:

```
npm run mutation:auth
```

Bước 2: Stryker tự động:

1. Đọc file code nguồn `authService.js`.
2. Tạo ra hàng chục mutant từ file đó.
3. Chạy bộ test auth với từng mutant một.
4. Ghi lại kết quả: Mutant nào bị diệt, mutant nào sống sót.

Bước 3: Stryker xuất báo cáo HTML:

- Hiển thị từng dòng code.
- Tô màu xanh (Killed) / đỏ (Survived).
- Chỉ ra chính xác bộ test còn thiếu assertion ở đâu.

Slide 11: Ví Dụ Thực Tế — Mutant Sống Sót Và Cách Diệt

Người phụ trách: 23127060 - Ninh Văn Khải

Mutant: Đổi điều kiện khóa tài khoản

Code gốc trong hàm đăng nhập:

```
// Kiểm tra tài khoản có đang bị khóa không
if (locked_until && today < locked_until) {
  return { error: "Tài khoản bị khóa" };
}
```

Mutant: Stryker đổi `&&` thành `||`:

```
if (locked_until || today < locked_until) { // lỗi logic
  return { error: "Tài khoản bị khóa" };
}
```

Kết quả: Người dùng bị khóa vĩnh viễn dù thời gian khóa đã hết.

Cách diệt mutant — Viết test case đặc thù:

```
test('cho phép đăng nhập khi thời gian khóa đã hết', async () => {
  // Đặt thời gian khóa ở 1 giây trước
```

Slide 12: AI Hỗ Trợ Việc Tìm Và Viết Test Như Thế Nào?

Người phụ trách: 23127060 - Ninh Văn Khải

Quy trình: Stryker tìm vấn đề → AI gợi ý giải pháp → Người kiểm chứng lại

Bước 1 — Stryker chạy và báo cáo:

"Mutant ở dòng 38: `[user.id]` đổi thành `[]` — Survived"

Bước 2 — Nhóm đọc báo cáo và hỏi AI:

"Stryker đổi `[user.id]` thành `[]` trong câu lệnh SQL. Bộ test không phát hiện ra. Hãy gợi ý cách viết test để diệt mutant này."

Bước 3 — AI gợi ý test case: AI phân tích code và đề xuất: Sau khi đăng nhập đúng, truy vấn database kiểm tra cột `login_attempts` có bằng 0 không.

Bước 4 — Nhóm tự chạy kiểm chứng lại: Nhập code do AI gợi ý, chạy `npm test` trên máy thật để xác nhận test PASS. Sau đó chạy lại Stryker để xác nhận mutant bị diệt.

Quan trọng: AI chỉ gợi ý ý tưởng. Con người phải tự chạy và kiểm chứng. AI có thể gợi ý sai.

Slide 13: Hoạt Động Thực Hành — "Kill The Mutant" (25 Phút)

Người phụ trách: 23127259 - Nguyễn Tấn Thắng

Cả lớp cùng thử diệt mutant

Cách chơi:

1. **(10 phút)** Nhóm 08 chiếu 5 mutant thực tế từ dự án. Mỗi mutant gồm:

- Code gốc.
- Code bị sửa (mutant).
- Bộ test hiện tại (đang không phát hiện được).

Nhiệm vụ: Mỗi nhóm viết thêm dòng `expect( ... )` để diệt mutant đó.

2. **(5 phút)** Các nhóm đổi bài cho nhau chấm chéo.

3. **(10 phút)** Nhóm 08 chạy thử assertion của từng nhóm **trực tiếp trên máy chiếu** bằng lệnh `npm test`.
Mutant bị diệt → Thắng!

Slide 14: Tổng Kết — 3 Bài Học Chính

Người phụ trách: Cả Nhóm 08 (23127195 - Trần Mạnh Hùng · 23127060 - Ninh Văn Khải · 23127259 - Nguyễn Tấn Thắng)

Sau bài học hôm nay, bạn cần nhớ 3 điều:

1. Test xanh chưa phải là test tốt.

Code Coverage chỉ đo "test có chạy qua không". Mutation Testing mới đo "test có phát hiện lỗi không".

2. Assertion cụ thể hơn = test tốt hơn.

Đừng chỉ viết `expect(status).toBe(200)`. Hãy kiểm tra thêm body, dữ liệu trả về, trạng thái database.

3. AI hỗ trợ nhanh hơn, nhưng con người phải kiểm chứng.

AI gợi ý test case tốt, nhưng có thể sai. Luôn phải tự chạy lại để xác nhận.

 **Video Demo:** <<DÁN_YOUTUBE_UNLISTED_LINK_TẠI_ĐÂY>>

Cảm ơn Thầy và các bạn đã theo dõi. Nhóm sẵn sàng nhận câu hỏi.